

EE 608

APPLIED MODELING AND OPTIMIZATION



Machine Learning Optimization for Finger Movement Classification Using XGBoost

Team Members:

- Jiajun Zhang
- Smit Mineshkumar Desai

GITHUB REPOSITORY

COURSE

INSTRUCTOR: Prof. K. P. Subbalakshmi

Contributions Of Each Team Member



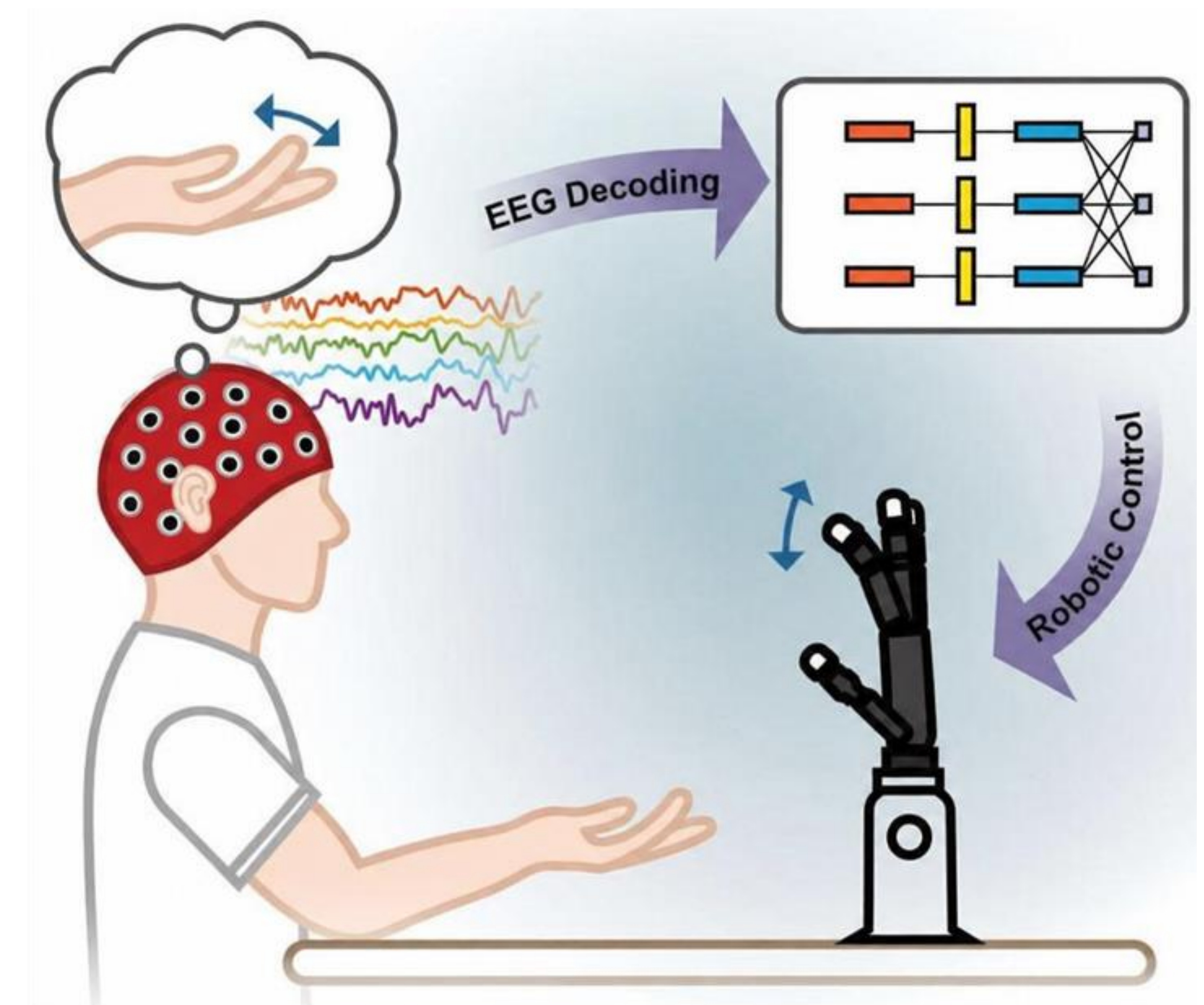
Jiajun Zhang: Designed and implemented the first-stage optimization pipeline for ECoG-based finger movement classification. This included signal windowing, movement threshold optimization, feature extraction, baseline XGBoost training, and evaluation using accuracy, macro-F1 score, classification report, and confusion matrix. Also contributed to the formulation of the optimization model and analysis of how preprocessing parameters affect classification performance.

Smit Desai: Developed and implemented the second-stage model optimization process. This included tuning XGBoost hyperparameters and regularization parameters such as max_depth, reg_alpha, reg_lambda, gamma, and min_child_weight. Also contributed to comparing baseline and optimized models, analyzing model complexity control, and interpreting the final optimization results.

Problem Description

Unlike the mature commercial products currently available—which utilize EMG-controlled robotic arms to assist individuals with limb loss in resuming a normal life—products that employ electroencephalography (EEG/ECoG) as an input signal to control robotic hands remain exceedingly rare. This scarcity persists even though such systems hold the potential to address cases where EMG signals cannot be acquired (e.g., in the absence of an upper arm). The primary reason for this lies in the inherent complexity of the human brain, which generates significant signal noise. In this project, we developed a model trained using XGBoost to classify finger movement states based on features extracted from electrocorticography (ECoG) signals; specifically, this encompasses the resting state as well as the opening and closing movements of various finger groups.

However, the initial performance results were quite limited, yielding an **accuracy of only 0.48** and a **Macro-F1 score of 0.28**. Consequently, **this project aims to optimize this model**. Specifically, we seek to enhance the overall performance of this multi-class classification task by optimizing parameters such as signal **window size**, movement **thresholds**, and the **hyperparameters** of the XGBoost model. Given that the dataset encompasses multiple movement categories and exhibits class imbalance, we have selected the Macro-F1 score as the primary evaluation metric for this project.



Optimization Model

The goal is to maximize the average cross-validation Macro-F1 score:

$$\max_{w, \alpha, \theta} CV-MacroF1(w, \alpha, \theta)$$

Divide by two part



Macro-F1 is used as the main objective because this is a 9-class classification problem with class imbalance. Accuracy alone may be misleading if the model predicts the dominant class too often. The window size and threshold are optimized because they directly affect signal segmentation and movement label quality. XGBoost hyperparameters are optimized because they affect model complexity, generalization, and classification performance.

Since the objective function is non-convex, non-smooth, and has no closed-form solution, numerical optimization methods are used. Grid search is used for w and α , and Bayesian optimization is used for XGBoost hyperparameters.

First Step: finding the best w^* and α^* (two-dimensional grid search) :

$$(w^*, \alpha^*) = \arg \max_{w, \alpha} \frac{1}{K} \sum_{k=1}^K MacroF1_k(w, \alpha)$$

Second Step: With the best w^* and α^* , find the best XGBoost hyperparameters (Bayesian optimization) :

$$\theta^* = \arg \max_{\theta} CV-MacroF1(w^*, \alpha^*, \theta)$$

$$w \in \{0.20, 0.25, 0.30, 0.50\}$$

$$\alpha \in \{0.15, 0.20, 0.25, 0.30, 0.35\}$$

$$\theta \in \Theta$$

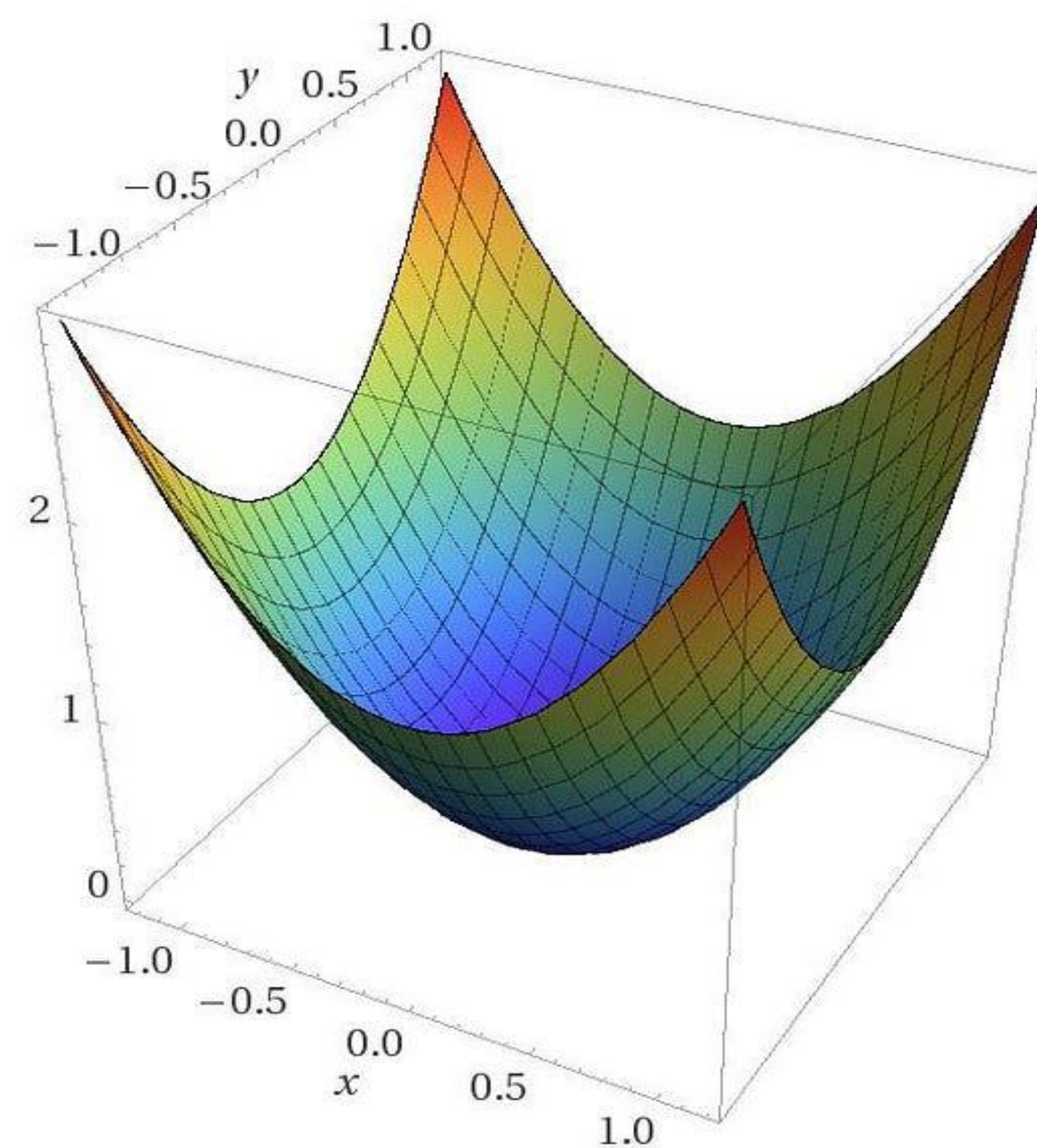
where Θ is the feasible set of XGBoost hyperparameters, such as `max_depth`, `learning_rate`, `n_estimators`, `gamma`, `reg_alpha`, and `reg_lambda`.

Proposed Solution

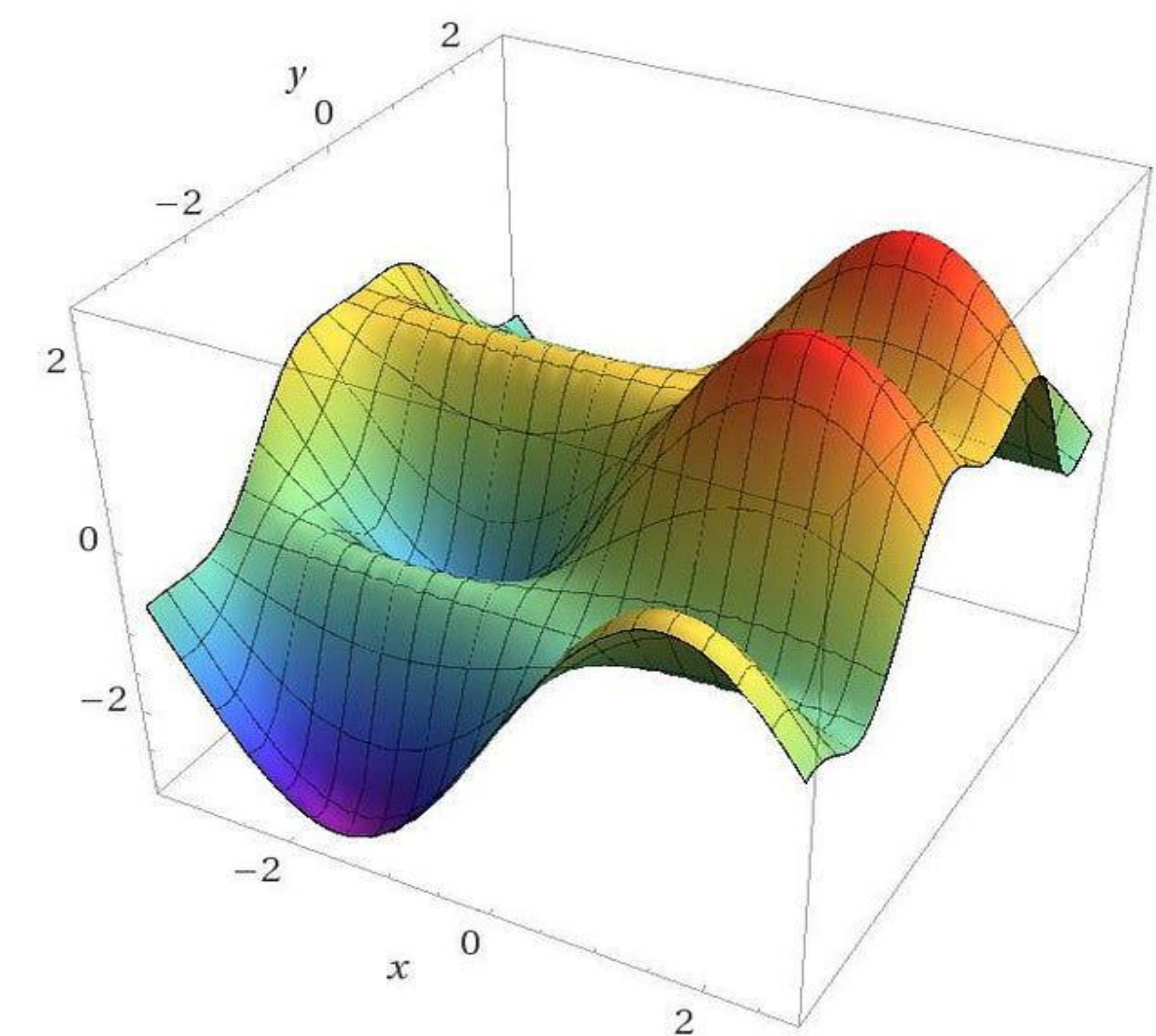
Convex or Non-convex?

- This 3-level optimization problem (window, threshold, hyperparameters) problem is **non-convex and non-smooth** because the objective is based on CV Macro-F1 after model training and prediction.
- Objective: maximize cross-validation CVMacro-F1:
- Because the objective depends on the full machine learning pipeline and includes discrete variables, it has no closed-form solution, so we solve it as a black-box numerical optimization problem.

$$CV-MacroF1(w, \alpha, \theta)$$



Computed by Wolfram|Alpha



Computed by Wolfram|Alpha

Proposed Solution



Feasible Set

For Stage 1, we optimize window size and movement threshold:

$$w \in \{0.20, 0.25, 0.30, 0.50\}$$

$$\alpha \in \{0.15, 0.20, 0.25, 0.30, 0.35\}$$

So the feasible set for Stage 1 is:

$$S_1 = \{(w, \alpha) : w \in W, \alpha \in A\}$$

For Stage 2, after selecting the best w^* and α^* , Bayesian optimization is used to search the feasible set of XGBoost hyperparameters:

$$\theta \in \Theta$$

where Θ includes allowed ranges for parameters such as **max_depth**, **learning_rate**, **n_estimators**, **gamma**, **reg_alpha**, and **reg_lambda**.

Top 10 optimization results ranked by CV Macro-F1:

	window_sec	alpha	cv_accuracy_mean	cv_accuracy_std	cv_macro_f1_mean	cv_macro_f1_std	num_windows	num_folds
0	0.50	0.20	0.521250	0.026398	0.399547	0.021819	800	5
1	0.30	0.15	0.503407	0.022618	0.399387	0.023001	1333	5
2	0.50	0.25	0.526250	0.032210	0.397880	0.034316	800	5
3	0.30	0.20	0.511653	0.030641	0.390908	0.040531	1333	5
4	0.30	0.30	0.523612	0.021910	0.376952	0.030286	1333	5
5	0.30	0.25	0.505643	0.018095	0.375869	0.016779	1333	5
6	0.50	0.35	0.523750	0.016956	0.375054	0.017324	800	5
7	0.50	0.15	0.502500	0.025800	0.374723	0.052311	800	5
8	0.25	0.20	0.495625	0.014334	0.367715	0.019796	1600	5
9	0.25	0.15	0.487500	0.018435	0.367148	0.032221	1600	5

Detailed per-class performance (combined):				
	precision	recall	f1-score	support
0	0.55	0.52	0.53	87
1	0.54	0.56	0.55	50
2	0.44	0.22	0.29	32
3	0.38	0.56	0.45	41
4	0.41	0.25	0.31	28
5	0.20	0.45	0.28	22
6	0.09	0.10	0.10	20
7	0.52	0.38	0.44	61
8	0.50	0.47	0.49	59
accuracy			0.43	400
macro avg	0.40	0.39	0.38	400
weighted avg	0.46	0.43	0.43	400

Proposed Solution



Why does a solution exist?

- **Stage 1 has a finite feasible set**
 - 4 window sizes $\times$ 5 threshold values = 20 combinations
 - Each (w, α) pair can be evaluated by Stratified K-Fold CV
- **Stage 2 also has a defined feasible set**
 - XGBoost hyperparameters are searched within predefined ranges
 - Bayesian optimization evaluates candidate parameter settings
- **Best observed solution can be selected**
 - We choose the combination with the highest mean CV Macro-F1

Is there a closed-form solution?

- **No closed-form solution**
 - The objective depends on the full ML pipeline:
 - signal windowing
 - label generation
 - feature extraction
 - XGBoost training
 - prediction and Macro-F1 evaluation
- **Reason**
 - The objective is non-smooth and non-differentiable
 - Some variables are discrete
- **Numerical solution**
 - Stage 1: Grid Search
 - Stage 2: Bayesian Optimization

Proposed Solution



Numerical Method / Toolbox

Stage 1: Grid Search

- Used to optimize:
 - window size w
 - threshold α
- Every (w, α) pair in the feasible set was tested.
- Each pair was evaluated using **Stratified K-Fold Cross Validation.**

Toolbox / Libraries

- NumPy / Pandas for data processing
- Scikit-learn for cross validation and evaluation metrics
- XGBoost for classification
- Bayesian optimization package such as Optuna / scikit-optimize

Stage 2: Bayesian Optimization

- Used to tune XGBoost hyperparameters:
 - max_depth
 - learning_rate
 - n_estimators
 - gamma
 - reg_alpha
 - reg_lambda
- This is more efficient than full grid search for a larger hyperparameter space.

Main optimization metric

- Mean CV Macro-F1 score

Validation — Dataset & Setup



Data source

- **BCI Competition IV, Dataset 4** (Miller & Schalk).
- Subject 1: 62 ECoG channels, 1000 Hz sampling, 0.15–200 Hz bandpass.
- Training: 400 s (400,000 samples); Test: 200 s (200,000 samples).
- Finger flexion captured by 5-finger data glove (super-sampled to 1000 Hz).

Window-level setup

- After windowing (window_sec = 0.25 baseline → 0.5 optimized): **800–1600 train windows / 400–800 test windows.**
- 558 statistical features per window (9 features × 62 channels).
- 9 movement classes after merging ring + little finger.

Held-out evaluation

- Test set never seen during optimization.
- All Optuna trials operated on training set only via 5-fold StratifiedKFold.

Validation — Cross-Validation as Our Truth Test

Equation:

$$\text{CV-Macro-F1} = \frac{1}{K} \sum_{k=1}^K \frac{1}{C} \sum_{c=1}^C \text{F1}_{c,k}$$

where $K = 5$ folds, $C = 9$ classes, and F1 of class c in fold k is:

$$\text{F1}_{c,k} = \frac{2 \cdot \text{precision}_{c,k} \cdot \text{recall}_{c,k}}{\text{precision}_{c,k} + \text{recall}_{c,k}}$$

Why this metric: -

- **Macro-F1** weights every class equally → rewards minority-class performance.
- **StratifiedKFold** preserves class distribution across folds → prevents biased splits.
- **5 folds** → each trial trains 5 models, averaging out lucky/unlucky splits.

Diagram suggestion: show 5 horizontal bars representing the dataset, each split into 4/5 train (blue) + 1/5 val (orange) with the orange block shifting position across the 5 bars.

Validation — Problems Faced & Mitigations



#	Problem	Mitigation
1	Validation overfitting — single-split Optuna picked configs that won by luck, not generalization.	Class imbalance — no_movement dominated 36% of windows; could score 36% by predicting "rest" always.
2	Class imbalance — no_movement dominated 36% of windows; could score 36% by predicting "rest" always.	Applied sample_weight="balanced" and used Macro-F1 as primary metric.
3	Data-vs-balance tradeoff — longer windows improved class balance but halved the training set (1600 → 800).	Acknowledged as fundamental limit; needs more subjects, not more tuning.

Key Insights — What Moved the Needle



The optimization hierarchy we discovered: -

- **Data pipeline tuning** (window size, threshold α) gave the biggest gain — ~6 pp accuracy improvement.
- **Hyperparameter tuning** alone gave only marginal gains (~1 pp) and risked validation overfitting.
- **CV + class weighting** balanced per-class performance — accuracy 48.6 → 47.6% but **Macro-F1 jumped 0.28 → 0.38**.
- The class-weighting tradeoff is the *right* one for BCI: catching rare finger actions matters more than just predicting "rest".

Takeaway: *Optimize the data before the model; in imbalanced safety-critical problems, balance > raw accuracy.*

Key Insights — Where the Ceiling Is



What the optimization revealed about fundamental limits..

- Three independent optimization passes (single-val Optuna, CV-Optuna, joint window+threshold+hyperparameters) all converged around **48–52% accuracy / 0.38–0.44 Macro-F1**.
- This convergence tells us hyperparameter tuning has hit its ceiling — further gains require *changing the problem*, not the optimizer.
- The real bottleneck is **information content**: 62 channels on one subject with statistical features can only carry so much movement-related signal.
- To break the ceiling: more subjects, frequency-domain features, or richer architectures (LSTM, transformer).

Takeaway: *When multiple optimization strategies plateau at the same point, you've found the data's information ceiling — not a tuning failure.*

Major Conclusions — Viability



Was our approach viable? **Yes, with caveats.**

- Manual baseline reached **48.6% accuracy** and **0.28 Macro-F1** on a 9-class problem — confirms the end-to-end pipeline works far above the 11% random baseline.
- Joint optimization (window + threshold + hyperparameters) improved **CV Macro-F1 from 0.34 → 0.44** — a meaningful gain from systematic tuning.
- The optimization story is honest: each layer (data pipeline, then hyperparameters) contributed, but ~48% is the real ceiling for single-subject, 9-class decoding with statistical features.

Major Conclusions — Difficulties



Problems we ran into and how we handled them:

- **Validation overfitting** — single-split Optuna picked configs that won by luck; fixed with 5-fold cross-validation.
- **Class imbalance** — `no_movement` dominated 36% of windows; required `class_weight='balanced'` sample weighting.
- **Numerical instability** — RMS computation triggered `sqrt` warnings on `int32` data; fixed by casting to `float64`.
- **Non-convex loss** — XGBoost has no global-optimum guarantee; different Optuna runs find different "best" configs.
- **Data-vs-balance tradeoff** — longer windows improved class balance but halved the training set, capping further gains.

Future Work



With more time, compute, and funding...

- **Bigger data** — combine all 3 BCI Competition patients with proper spatial electrode alignment; augment with Stanford ECoG Library.
- **Better models** — per-subject training, LSTM/transformer for temporal context, frequency-domain features (high-gamma band power).
- **Working prototype** — deploy on Arduino-controlled robotic hand for real-time closed-loop BCI demo.
- **Path to billions** — partner with hospitals for clinical trials, patent the optimization pipeline, compete with Neuralink in the assistive BCI market.